

• [BLOCKS OF FILE SYSTEM]

Unix file system consists of four blocks:

1. Boot Block
2. Super Block
3. I-NODE Block
4. Data Block

Each of which generally 512 Bytes. Some file system may have 1024 or 2048 Bytes. The block size depends upon how file system can be implemented particularly installation. It may vary from version to version.

• **Boot Block** → Contains boot strap loaders. Then referred as MBR (master boot record). This program is loaded into memory when the system is booted. It may intercom, load another program from the disk. But eventually it load the kernel into the memory. Although the boot block is needed to startup the system. All the file system contains one boot block.

• **Super Block** :- Boot block is followed by the Super Block.

It contains global file information about disk uses and availability of data block and I-NODE. It also contains:-

- (i) Size of the file system.
- (ii) Block size used by the file system.
- (iii) No of ~~free~~ data block available and Perential
Free
^

list immediately allocable for free data ~~blocks~~ blocks.

- (iv) No. of free INODE Available and Partial list of immediately usable INODE.
- (v) Last time of updating.
- (vi) The state of the file system that kernel maintainance.

INODE BLOCK : - It contains all INODE or ^{Index} ~~Information~~ ^{Information} NODE. When the file is created, and the INODE is allocated here even though the INODE contains all file attributes (except its name).

Each entry is made of 64 bytes and contain relevant details for that file. INODE Have :-

- (i) Owner of the file.
- (ii) The group to which the owner belong.
- (iii) Type of file.
- (iv) File access permission.
- (v) Date and time of the last access.
- (vi) Date & time of the last modification.
- (vii) No. of links to the file.
- (viii) Size of the file.
- (ix) ~~Address~~ Address of Blocks where the file is physically present.

Syntax :-

\$ ls -l filename

- **Data Block** → This block, contains actual file contents. Allocated block can belong to only one file in the file system. This block cannot be used for storing of any other file content unless the file to which originally belong or deleted.

Storage of file

Typical
INODE
Entry

	Owner
	Group
	File Type
	Permission
	Access file
	modification Time
	INODE modification Time
0	
1	
2	
3	
4	
5	
11	
12	

1 KB

1 KB

Explain Zombie State. (5)

→ when a process is in running state it may invoke this input output operation when it has nothing to do except wait for input-output for completion. Then the process move to sleeping state to be waiting up when input output operation is over.

Process can be suspended by the key 'ctrl + Z'.

Process whose parent do not wait for their died child.

when the child process dies then ~~the~~ parent process immediately move to the Zombie state. It remains in the state until its parent picks up the child's exit status. (The reason for waiting)

from the process table when that is done kernel frees the died child but you cannot kill it.

It also ~~possible~~ possible for parents itself to ^{die} ~~repent~~ the child's die. child then become ^{before} orphan and kernel makes 'init' process to kill all the orphan process. This state is called orphan state.

Syntax :- \$ kill

- How to make a foreground process into background process. ~~@\$!bg~~

→ \$bg

- How a lower priority process get higher priority.

→ \$nice

- write the mechanism of process creation. (5)

mechanism of process creation.

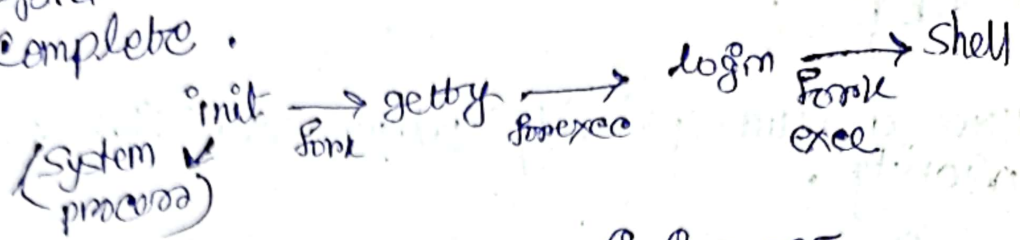


• **fork** → fork means creation of child.

- process in Unix is created.
- process image is practically identical to that of calling process except few parameters like pid.
- It gets new pid after forked the child.

• **exec** → fork creates a process, but it is not enough to run a new program. To do that forked child needs to overwrite its own image with the code and data of new program. (no new process is created).

- **Wait** → Parent have to executed wait System call for the child process to complete.



init → Demon Process as and for etc.

- what do you mean by Demon Process.

→ It is a background process that runs independently of any users control to perform specific system level task.

Example:- sshd

~~httpd~~ httpd

crond

syslogd

Note: If we write '&' sign after any command in unix then it will work as a background process.

- **nohup** → It permits the background process running while shell logout.
- write a script that would display the time when run.

→ #!/bin/sh

echo "The current date is 'date'"

exit ()

• WAP to verify how many users logged into the system.

→ #! bin / bin/sh

echo "There are 'who | wc -l' users in the system"

• All files starting with f ?

→ ls -l f *